

Nama:Hafidza Tsamara Zahra
NIM:254107020034
Kelas:1G-TI
Absen:11

LAPORAN TUGAS PERTEMUAN KE-7

6.2. Searching/ Pencarian Menggunakan Algoritma Sequential Search

6.2.1. Langkah-langkah Percobaan Sequential Search

class mahasiswaBerprestasi11

```
package praktikum05;

public class mahasiswaBerprestasi11 {
    mahasiswa11 [] listMhs= new mahasiswa11 [5];
    int idx;

    void tambah(mahasiswa11 m){
        if(idx<listMhs.length){
            listMhs[idx]=m;
            idx++;
        }else{
            System.out.println("data sudah penuh");
        }
    }

    void tampil(){
        for(mahasiswa11 m:listMhs){
            m.tampilInformasi();
            System.out.println("-----");
        }
    }

    void bubblesort(){
        for(int i=0;i<listMhs.length-1;i++){
            for(int j=1;j<listMhs.length-i;j++){
                if(listMhs[j].ipk>listMhs[j-1].ipk){
                    mahasiswa11 tmp = listMhs[j];
                    listMhs[j]=listMhs[j-1];
                    listMhs[j-1]=tmp;
                }
            }
        }
    }
}
```

```

    }

    void selectionSort() {
        for(int i=0;i<listMhs.length-1;i++){
            int idxMin=i;
            for(int j=i+1;j<listMhs.length;j++){
                if(listMhs[j].ipk<listMhs[idxMin].ipk){
                    idxMin=j;
                }
            }
            mahasiswa tmp = listMhs[idxMin];
            listMhs[idxMin]=listMhs[i];
            listMhs[i]=tmp;
        }
    }

    void insertionSort() {
        for(int i=1;i<listMhs.length;i++){
            mahasiswa temp = listMhs[i];
            int j=i;
            while(j>0 && listMhs[j-1].ipk < temp.ipk){
                listMhs[j]=listMhs[j-1];
                j--;
            }
            listMhs[j]=temp;
        }
    }

    int sequentialSearching(double cari) {
        int posisi = -1;
        for(int j=0;j<listMhs.length;j++){
            if(listMhs[j].ipk==cari) {
                posisi=j;
                break;
            }
        }
        return posisi;
    }

    void tampilPosisi(double x,int pos) {
        if (pos!=-1){
            System.out.println("data mahasiswa dengan ipk: " +x+ "
ditemukan pada indeks " +pos);
        }else{

```

```

        System.out.println("data " +x+ "tidak ditemukan");
    }
}

void tampilDataSearch(double x,int pos){
    if(pos !=-1){
        System.out.println("nim\t: "+listMhs[pos].nim);
        System.out.println("nama\t: "+listMhs[pos].nama);
        System.out.println("kelas\t: "+listMhs[pos].kelas);
        System.out.println("ipk\t: "+x);
    }else{
        System.out.println("data mahasiswa dengan ipk " +x+ "tdak
ditemukan") ;
    }
}
}
}

```

class mahasiswaDemo11

```

package praktikum05;

import java.util.Scanner;

public class mahasiswaDemo11 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        mahasiswaBerprestasi11 list = new mahasiswaBerprestasi11();

        int jumMhs=5;
        // System.out.print("Masukkan jumlah mahasiswa: ");
        // int jumlah = sc.nextInt();
        // sc.nextLine();

        for(int i=0; i<jumMhs; i++){
            System.out.println("Data mahasiswa ke-" + (i+1));

            System.out.print("NIM: ");
            String nim = sc.nextLine();

            System.out.print("Nama: ");
            String nama = sc.nextLine();

            System.out.print("Kelas: ");
            String kelas = sc.nextLine();

```

```

        System.out.print("IPK: ");
        double ipk = sc.nextDouble();
        sc.nextLine();

System.out.println("-----");
        mahasiswa11 m = new mahasiswa11(nim, nama, kelas, ipk);
        list.tambah(m);
    }

    // System.out.println("\nData sebelum sorting:");
    // list.tampil();

    // System.out.println("\nData setelah sorting (DESC):");
    // list.bubblesort();
    // list.tampil();

    // System.out.println("data yang sudah terurut menggunakan
SELECTION SORT (ASC)");
    // list.selectionSort();
    // list.tampil();

    // System.out.println("data yang sudah terurut menggunakan
INSERTION SORT (ASC)");
    // list.insertionSort();
    // list.tampil();

    list.tampil();

System.out.println("-----");
    System.out.println("pencarian data");

System.out.println("-----")
;

    System.out.println("masukkan ipk mahasiswa yang dicari: ");
    System.out.print("IPK: ");
    double cari = sc.nextDouble();

    System.out.println("menggunakan sequential searching");
    double posisi = list.sequentialSearching(cari);
    int pss= (int)posisi;
    list.tampilPosisi(cari,pss);
    list.tampilDataSearch(cari,pss);
}

```

```
}
```

hasil output

Data mahasiswa ke-1

NIM: 111

Nama: adi

Kelas: 2

IPK: 3.6

Data mahasiswa ke-2

NIM: 222

Nama: tio

Kelas: 2

IPK: 3.8

Data mahasiswa ke-3

NIM: 333

Nama: ila

Kelas: 2

IPK: 3.0

Data mahasiswa ke-4

NIM: 444

Nama: lia

Kelas: 2

IPK: 3.5

Data mahasiswa ke-5

NIM: 555

Nama: fia

Kelas: 2

IPK: 3.3

Nama: adi

NIM: 111

Kelas 2

IPK 3.6

Nama: tio

NIM: 222

Kelas 2

IPK 3.8

Nama: ila

NIM: 333

Kelas 2

IPK 3.0

Nama: lia

NIM: 444

Kelas 2

IPK 3.5

Nama: fia

NIM: 555

Kelas 2

IPK 3.3

```
-----  
pencarian data  
-----
```

```
masukkan ipk mahasiswa yang dicari:
```

```
IPK: 3.5
```

```
menggunakan seuential searching
```

```
data mahasiswa dengan ipk: 3.5ditemukan pada indeks3
```

```
nim      : 444
```

```
nama     : lia
```

```
kelas    : 2
```

```
ipk      : 3.5
```

```
PS E:\hafidza\algoritma-dan-struktur-data> █
```

6.2.3. Pertanyaan

1. Jelaskan perbedaan metod tampilDataSearch dan tampilPosisi pada class MahasiswaBerprestasi!
2. Jelaskan fungsi break pada kode program di bawah ini!

```
if(listMhs[j].ipk==cari) {  
    posisi=j;  
    break;  
}
```

3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?
4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.
5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?

jawaban

1. -tampilPosisi

Digunakan untuk menampilkan posisi (indeks) dari data yang ditemukan dalam array. Outputnya hanya memberi tahu *di index ke berapa* data berada.

-tampilDataSearch

Digunakan untuk menampilkan detail lengkap data mahasiswa yang ditemukan, seperti NIM, nama, kelas, dan IPK.

Kesimpulan:

tampilPosisi : hanya posisi data

tampilDataSearch : isi/data lengkap mahasiswa

2. Fungsi break adalah untuk menghentikan perulangan (loop) secara langsung saat data yang dicari sudah ditemukan.

Jadi Saat IPK ketemu langsung berhenti cari, Program tidak lanjut mengecek data berikutnya

3. Variabel pos digunakan untuk:
 - Menyimpan posisi/index data yang ditemukan
 - Menjadi acuan untuk:
 - Menampilkan posisi (tampilPosisi)
 - Mengambil data dari array (tampilDataSearch)Tanpa pos, program tidak tahu data mana yang ditemukan.
4. Sequential Search pada program ini akan menampilkan:
data yang pertama kali ditemukan (indeks paling kecil)
Karena:
 - Loop berjalan dari depan ke belakang
 - Ada break → langsung berhenti saat ketemu pertama
5. Jika break dihapus, maka:
 - Program akan tetap mencari sampai akhir array
 - Nilai posisi akan terus diperbaruiAkibatnya Yang ditampilkan adalah data terakhir yang ditemukan, bukan yang pertama

6.3. Searching/ Pencarian Menggunakan Algoritma Binary Search

6.3.1. Langkah-langkah Percobaan Binary Search

class mahasiswaBerprestasi11

```
package praktikum05;

public class mahasiswaBerprestasi11 {

    mahasiswa11 [] listMhs= new mahasiswa11 [5];

    int idx;

    void tambah(mahasiswa11 m) {

        if(idx<listMhs.length) {

            listMhs[idx]=m;
        }
    }
}
```

```
        idx++;

    }else{

        System.out.println("data sudah penuh");

    }

}

void tampil() {

    for(mahasiswa11 m:listMhs) {

        m.tampilInformasi();

        System.out.println("-----");

    }

}

void bubblesort() {

    for(int i=0;i<listMhs.length-1;i++){

        for(int j=1;j<listMhs.length-i;j++){

            if(listMhs[j].ipk>listMhs[j-1].ipk) {

                mahasiswa11 tmp = listMhs[j];

                listMhs[j]=listMhs[j-1];

                listMhs[j-1]=tmp;

            }

        }

    }

}
```

```
void selectionSort(){

    for(int i=0;i<listMhs.length-1;i++){

        int idxMin=i;

        for(int j=i+1;j<listMhs.length;j++){

            if(listMhs[j].ipk<listMhs[idxMin].ipk){

                idxMin=j;

            }

        }

        mahasiswa tmp = listMhs[idxMin];

        listMhs[idxMin]=listMhs[i];

        listMhs[i]=tmp;

    }

}
```

```
void insertionSort(){

    for(int i=1;i<listMhs.length;i++){

        mahasiswa temp = listMhs[i];

        int j=i;

        while(j>0 && listMhs[j-1].ipk < temp.ipk){

            listMhs[j]=listMhs[j-1];

            j--;

        }

        listMhs[j]=temp;

    }

}
```

```
int sequentialSearching(double cari){

    int posisi = -1;

    for(int j=0;j<listMhs.length;j++){

        if(listMhs[j].ipk==cari){

            posisi=j;

            break;

        }

    }

    return posisi;

}

void tampilPosisi(double x,int pos){

    if (pos!=-1){

        System.out.println("data mahasiswa dengan ipk: " +x+ " "
ditemukan pada indeks " +pos);

    }else{

        System.out.println("data" +x+ "tidak ditemukan");

    }

}

void tampilDataSearch(double x,int pos){

    if(pos !=-1){

        System.out.println("nim\t: "+listMhs[pos].nim);

        System.out.println("nama\t: "+listMhs[pos].nama);

        System.out.println("kelas\t: "+listMhs[pos].kelas);

        System.out.println("ipk\t: "+x);

    }

}
```

```

        }else{

            System.out.println("data mahasiswa dengan ipk " +x+ "tdak
ditemukan") ;

        }

    }

    int findBinarySearch(double cari,int left,int right){

        int mid;

        if(right>=left){

            mid =(left+right)/2;

            if(cari ==listMhs[mid].ipk){

                return (mid);

            }else if (listMhs[mid].ipk>cari){

                return findBinarySearch(cari, left, mid-1);

            }else{

                return findBinarySearch(cari, mid+1, right);

            }

        }

        return -1;

    }

}

```

class mahasiswaDemo11

```

package praktikum05;

import java.util.Scanner;

```

```
public class mahasiswaDemo11 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        mahasiswaBerprestasi11 list = new mahasiswaBerprestasi11();

        int jumMhs=5;

        // System.out.print("Masukkan jumlah mahasiswa: ");

        // int jumlah = sc.nextInt();

        // sc.nextLine();

        for(int i=0; i<jumMhs; i++){

            System.out.println("Data mahasiswa ke-" + (i+1));

            System.out.print("NIM: ");

            String nim = sc.nextLine();

            System.out.print("Nama: ");

            String nama = sc.nextLine();

            System.out.print("Kelas: ");

            String kelas = sc.nextLine();

            System.out.print("IPK: ");

            double ipk = sc.nextDouble();
```

```
        sc.nextLine();

System.out.println("-----");

        mahasiswa11 m = new mahasiswa11(nim, nama, kelas, ipk);

        list.tambah(m);

    }

    // System.out.println("\nData sebelum sorting:");

    // list.tampil();

    // System.out.println("\nData setelah sorting (DESC):");

    // list.bubblesort();

    // list.tampil();

    // System.out.println("data yang sudah terurut menggunakan
SELECTION SORT (ASC)");

    // list.selectionSort();

    // list.tampil();

    // System.out.println("data yang sudah terurut menggunakan
INSERTION SORT (ASC)");

    // list.insertionSort();

    // list.tampil();

    list.tampil();

System.out.println("-----");
```

```

        System.out.println("pencarian data");

System.out.println("-----");
;

        System.out.println("masukkan ipk mahasiswa yang dicari: ");

        System.out.print("IPK: ");

        double cari = sc.nextDouble();

        // System.out.println("menggunakan sequential searching");

        // double posisi = list.sequentialSearching(cari);

        // int pss= (int)posisi;

        // list.tampilPosisi(cari,pss);

        // list.tampilDataSearch(cari,pss);

        System.out.println("-----");

        System.out.println("menggunakan binary search");

        System.out.println("-----");

        double posisi2 = list.findBinarySearch(cari, 0, jumMhs-1);

        int pss2 = (int)posisi2;

        list.tampilPosisi(cari,pss2);

        list.tampilDataSearch(cari,pss2);

    }
}

```

hasil output

Data mahasiswa ke-1

NIM: 111

Nama: adi

Kelas: 2

IPK: 3.1

Data mahasiswa ke-2

NIM: 222

Nama: ila

Kelas: 2

IPK: 3.2

Data mahasiswa ke-3

NIM: 333

Nama: lia

Kelas: 2

IPK: 3.3

Data mahasiswa ke-4

NIM: 444

Nama: susi

Kelas: 2

IPK: 3.5

Data mahasiswa ke-5

NIM: 555

Nama: anita

Kelas: 2

IPK: 3.7

Nama: adi

NIM: 111

Kelas 2

IPK 3.1

Nama: ila

NIM: 222

Kelas 2

IPK 3.2

Nama: lia

NIM: 333

Kelas 2

IPK 3.3

Nama: susi

NIM: 444

Kelas 2

IPK 3.5

Nama: anita

NIM: 555

Kelas 2

IPK 3.7

```
-----  
pencarian data  
-----
```

```
masukkan ipk mahasiswa yang dicari:
```

```
IPK: 3.7  
-----
```

```
menggunakan binary search  
-----
```

```
data mahasiswa dengan ipk: 3.7 ditemukan pada indeks 4
```

```
nim      : 555
```

```
nama     : anita
```

```
kelas    : 2
```

```
ipk      : 3.7  
-----
```

6.3.3. Pertanyaan

1. Tunjukkan pada kode program yang mana proses divide dijalankan!
2. Tunjukkan pada kode program yang mana proses conquer dijalankan!
3. Apa fungsi left, right, dan mid?
4. Jika data IPK yang dimasukkan tidak urut. Apakah program masih dapat berjalan? Mengapa demikian?
5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary search agar hasilnya sesuai
6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.
7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard

jawaban

1.

```
mid = (left+right) / 2;
```

Di sini array dibagi menjadi dua bagian:

-kiri : left sampai mid-1

-kanan : mid+1 sampai right

2.

```
else if (listMhs[mid].ipk > cari) {  
    return findBinarySearch(cari, left, mid-1);  
} else {
```

```
        return findBinarySearch(cari, mid+1, right);  
    }  
}
```

Jika nilai tengah lebih besar maka cari ke kiri

Jika lebih kecil maka cari ke kanan

3. left : batas kiri (awal array / bagian yang dicari)

right : batas kanan (akhir array / bagian yang dicari)

mid : indeks tengah hasil pembagian $(\text{left} + \text{right}) / 2$

4. Program tetap jalan, tapi hasilnya salah

Karena: Binary Search harus data terurut (ascending/descending), Kalau tidak urut maka pembagian kiri/kanan jadi tidak valid.

- 5.

```
int findBinarySearch(double cari, int left, int right) {  
    int mid;  
    if(right >= left) {  
        mid = (left + right) / 2;  
        if(cari == listMhs[mid].ipk) {  
            return mid;  
        } else if (listMhs[mid].ipk < cari) {  
            return findBinarySearch(cari, left, mid-1);  
        } else {  
            return findBinarySearch(cari, mid+1, right);  
        }  
    }  
    return -1;  
}
```

6. dilihat dari kondisi

```
if(right >= left) {
```

dan

```
return -1;
```

Artinya:

-Jika $\text{right} < \text{left}$ maka pencarian berhenti

-Program mengembalikan -1 tanda data tidak ditemukan

7. modifikasi di class mahasiswaDemo11

```
package praktikum05;  
  
import java.util.Scanner;
```

```
public class mahasiswaDemol1 {  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Masukkan jumlah mahasiswa: ");  
  
        int jumMhs = sc.nextInt();  
  
        sc.nextLine();  
  
        mahasiswaBerprestasi11 list = new  
mahasiswaBerprestasi11();  
  
        list.listMhs = new mahasiswa11[jumMhs];  
  
        for(int i=0; i<jumMhs; i++){  
  
            System.out.println("Data mahasiswa ke-" + (i+1));  
  
            System.out.print("NIM: ");  
  
            String nim = sc.nextLine();  
  
            System.out.print("Nama: ");  
  
            String nama = sc.nextLine();  
  
            System.out.print("Kelas: ");  
  
            String kelas = sc.nextLine();  
  
            System.out.print("IPK: ");  
  
            double ipk = sc.nextDouble();  

```

```
        sc.nextLine();

System.out.println("-----");

        mahasiswa1 m = new mahasiswa1(nim, nama, kelas,
ipk);

        list.tambah(m);

    }

    // System.out.println("\nData sebelum sorting:");

    // list.tampil();

    // System.out.println("\nData setelah sorting (DESC):");

    // list.bubblesort();

    // list.tampil();

    // System.out.println("data yang sudah terurut
menggunakan SELECTION SORT(ASC)");

    // list.selectionSort();

    // list.tampil();

    // System.out.println("data yang sudah terurut
menggunakan INSERTION SORT (ASC)");

    // list.insertionSort();

    // list.tampil();

    list.tampil();
```

```

System.out.println("-----
---");

        System.out.println("pencarian data");

System.out.println("-----
----");

        System.out.println("masukkan ipk mahasiswa yang dicari:
");

        System.out.print("IPK: ");

        double cari = sc.nextDouble();

        // System.out.println("menggunakan sequential
searching");

        // double posisi = list.sequentialSearching(cari);

        // int pss= (int)posisi;

        // list.tampilPosisi(cari,pss);

        // list.tampilDataSearch(cari,pss);


        System.out.println("-----");

        System.out.println("menggunakan binary search");

        System.out.println("-----");

        double posisi2 = list.findBinarySearch(cari, 0,
jumMhs-1);

        int pss2 = (int)posisi2;

        list.tampilPosisi(cari,pss2);

        list.tampilDataSearch(cari,pss2);

    }

```

```
}
```

hasil output

Masukkan jumlah mahasiswa: 5

Data mahasiswa ke-1

NIM: 111

Nama: adi

Kelas: 2

IPK: 3.1

Data mahasiswa ke-2

NIM: 222

Nama: ila

Kelas: 2

IPK: 3.2

Data mahasiswa ke-3

NIM: 333

Nama: lia

Kelas: 2

IPK: 3.5

Data mahasiswa ke-4

NIM: 444

Nama: susi

Kelas: 2

IPK: 3.3

Data mahasiswa ke-5

NIM: 555

Nama: anita

Kelas: 2

IPK: 3.7

data yang sudah terurut menggunakan SELECTION SORT(ASC)

Nama: adi

NIM: 111

Kelas 2

IPK 3.1

Nama: ila

NIM: 222

Kelas 2

IPK 3.2

Nama: susi

NIM: 444

Kelas 2

IPK 3.3

Nama: lia

NIM: 333

Kelas 2

IPK 3.5

Nama: anita

NIM: 555

Kelas 2

IPK 3.7

pencarian data

masukkan ipk mahasiswa yang dicari:

IPK: 3.7

menggunakan binary search

data3.7tidak ditemukan

data mahasiswa dengan ipk 3.7tdak ditemukan